The Wayback Machine - https://web.archive.org/web/20220719145528/https://vuls.cert.org/confluence/display/Wiki/2021/06/21/Finding+Privilege+Escalation+Vulnerab…

Finding Privilege Escalation Vulnerabilities in Windows using Process Monitor

Created by Will Dormann, last modified on 2022-06-14

- Overview
- The Concept
- What to look for
- Using the filter
- Investigating results
- Mistakes that developers make
  - Unexpected paths being accessed
    - URL-encoded paths
    - POSIX paths
    - Use of a library that loads from an unexpected path
    - Use of paths that only exist on a developer's system
  - Unexpected ACLs applied to paths being used
    - Using the C:\ProgramData\ directory without explicitly settings ACLs
    - Installing to a subdirectory off of the system root
    - Allowing user-specified installation directories without settings ACLs
- Defenses against privilege escalation
  - Remove "Create folders" permission on system root for unprivileged users
  - Do not install software outside of C:\Program Files\
  - Test and fortify your own systems

Overview

This post will explain how to find privilege escalation vuls on Windows that no one appears to be looking for, because it's been pretty easy to find a bunch of them. After explaining how to find them, I'll introduce some defenses that can partly mitigate the problem in different ways. But what I'd like to see change is for developers to start looking for these vuls in the way I describe so that they stop introducing them in the first place.

Back when we first released CERT BFF, the usual process for putting together a proof-of-concept exploit for a memory corruption vulnerability was:

1. Fuzz the target until you get control of the instruction pointer.
2. Find out which bytes can be used to store your shellcode, using BFF string minimization.
3. Use ROP as necessary to modify the program flow so that it executes your shellcode.

It was often relatively straightforward to go from Start to PoC with CERT BFF. As time went on, the bar for exploiting memory corruption vulnerabilities was raised. This can likely be attributed to two things that happened over the years:

1. Increased fuzzing by parties releasing software.
2. Increased presence of exploit mitigations in both software and the platforms that they run on.

I have recently worked on a vulnerability discovery technique that reminded me of the early BFF days. Both with respect to how easy it is to find the vulnerabilities and also how easy it can be to exploit them. In fact, the concept is so trivial that I was surprised by how successful it was in finding vulnerabilities. Just like the idea of going directly from fuzzing with BFF to a working exploit became less and less viable as time went on, I'd like for there to be much less low-hanging fruit that can be easily found with this technique.

In this post I will share some of my findings as well as the filter itself for finding privilege escalation vulnerabilities with Sysinternals Process Monitor (Procmon).

The Concept

When software is installed on the Windows platform, some components of it may run with privileges, regardless of which user is currently logged on to the system. These privileged components generally take two forms:

1. Installed services
2. Scheduled tasks

How might we achieve privilege escalation on a Windows system? Any time that a privileged process interacts with a resource that an unprivileged user may be able to influence, this opens up the possibility for a privilege escalation vulnerability.

What to look for

The easiest way to check for privileged processes that might be able to be influenced by non-privileged users is to use a Process Monitor filter that displays operations based on the following attributes:

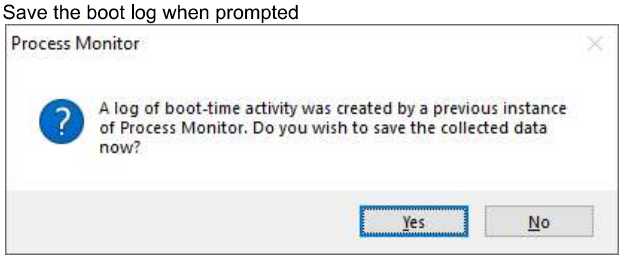
1. Files or directories that do not exist.
2. Processes that have elevated privileges.
3. Locations that may be writable by an unprivileged user.

Checks 1 and 2 can be trivially implemented in Process Monitor. Check 3 is a little more complicated and may result in some false positives if we limit our tool to strictly what can be done with a Process Monitor Filter. But I've created a filter [Download from Github] that seems to do a pretty good job of making privilege escalation vulnerabilities pretty obvious.

Using the filter

Using the Privesc.PMF Process Monitor filter is relatively straightforward:

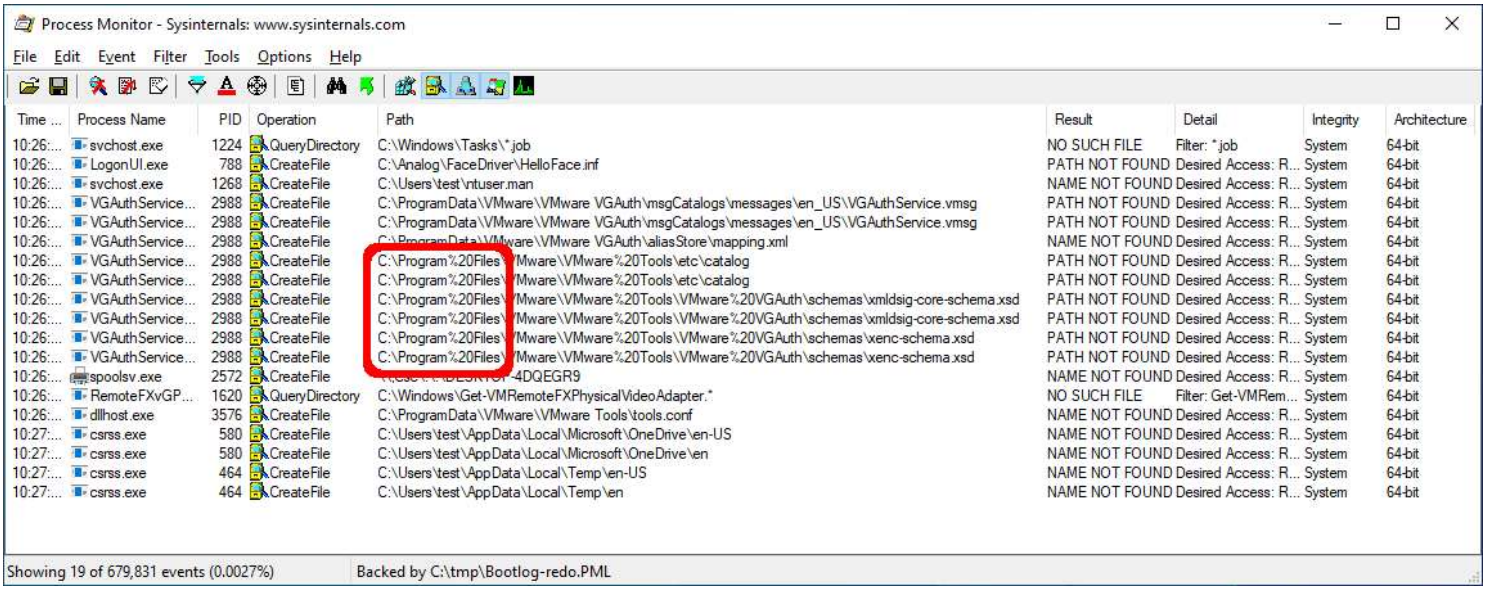
1. Enable Process Monitor boot logging (Options → Enable Boot Logging)
2. Reboot and log in
3. Run Process Monitor
4. Save the boot log when prompted



5. Import the "Privesc" filter (Filter → Organize Filters → Import...)
6. Apply the Privesc filter (Filter → Load Filter → Privesc)
7. Look for and investigate unexpected file accesses.

Investigating results

Let's start by looking at a boot log of a common baseline that we might deal with as a vulnerability analyst - a 64-bit Windows 10 2004 system with VMware Tools installed:



Time ...	Process Name	PID	Operation	Path	Result	Detail	Integrity	Architecture
10:26:...	svchost.exe	1224	QueryDirectory	C:\Windows\Tasks*.job	NO SUCH FILE	Filter: *.job	System	64-bit
10:26:...	LogonUI.exe	788	CreateFile	C:\Analog\FaceDriver\HelloFace.inf	PATH NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	svchost.exe	1268	CreateFile	C:\Users\test\ntuser.man	NAME NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	VGAuthService...	2988	CreateFile	C:\ProgramData\VMware\VMware VGAuth\msgCatalogs\messages\en_US\VGAuthService.vmsg	PATH NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	VGAuthService...	2988	CreateFile	C:\ProgramData\VMware\VMware VGAuth\msgCatalogs\messages\en_US\VGAuthService.vmsg	PATH NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	VGAuthService...	2988	CreateFile	C:\ProgramData\VMware\VMware VGAuth\aliasStore\mapping.xml	NAME NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	VGAuthService...	2988	CreateFile	C:\Program%20Files\VMware\VMware%20Tools\etc\catalog	PATH NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	VGAuthService...	2988	CreateFile	C:\Program%20Files\VMware\VMware%20Tools\etc\catalog	PATH NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	VGAuthService...	2988	CreateFile	C:\Program%20Files\VMware\VMware%20Tools\VMware%20VGAuth\schemas\xmldsig-core-schema.xsd	PATH NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	VGAuthService...	2988	CreateFile	C:\Program%20Files\VMware\VMware%20Tools\VMware%20VGAuth\schemas\xmldsig-core-schema.xsd	PATH NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	VGAuthService...	2988	CreateFile	C:\Program%20Files\VMware\VMware%20Tools\VMware%20VGAuth\schemas\xenc-schema.xsd	PATH NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	VGAuthService...	2988	CreateFile	C:\Program%20Files\VMware\VMware%20Tools\VMware%20VGAuth\schemas\xenc-schema.xsd	PATH NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	spoolsv.exe	2572	CreateFile	[illegible]-4DQEGR9	NAME NOT FOUND	Desired Access: R...	System	64-bit
10:26:...	RemoteFXvGP...	1620	QueryDirectory	C:\Windows\Get-VMRemoteFXPhysicalVideoAdapter.*	NO SUCH FILE	Filter: Get-VMRem...	System	64-bit
10:26:...	dllhost.exe	3576	CreateFile	C:\ProgramData\VMware\VMware Tools\tools.conf	NAME NOT FOUND	Desired Access: R...	System	64-bit
10:27:...	csrss.exe	580	CreateFile	C:\Users\test\AppData\Local\Microsoft\OneDrive\en-US	NAME NOT FOUND	Desired Access: R...	System	64-bit
10:27:...	csrss.exe	580	CreateFile	C:\Users\test\AppData\Local\Microsoft\OneDrive\en	NAME NOT FOUND	Desired Access: R...	System	64-bit
10:27:...	csrss.exe	464	CreateFile	C:\Users\test\AppData\Local\Temp\en-US	NAME NOT FOUND	Desired Access: R...	System	64-bit
10:27:...	csrss.exe	464	CreateFile	C:\Users\test\AppData\Local\Temp\en	NAME NOT FOUND	Desired Access: R...	System	64-bit

Even with virtually no software installed in our VM, we can already see something suspicious: **C:\Program%20Files**

Windows users may be familiar with the path **C:\Program Files**, but what's with the **%20**? Why might such a file operation occur? We'll cover the reason in the section below.

Mistakes that developers make

There are a number of mistakes that a developer might make that can lead to a privileged process being able to be influenced by an unprivileged user. The mistakes that I've noticed with respect to simple privilege escalation vulnerabilities with Windows applications fall into two main categories:

1. Unexpected paths being accessed.
2. Unexpected Access Control Lists (ACLs) applied to paths being used.

Unexpected paths being accessed

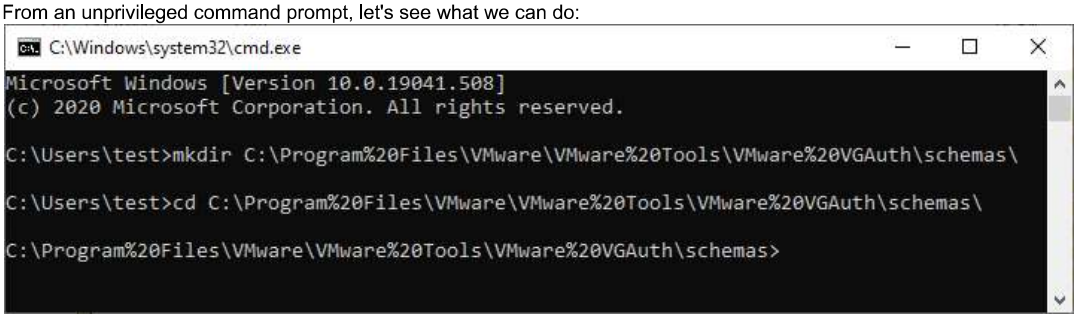
In some cases, an unexpected path is accessed during the execution of a program. That is, the developer would probably be surprised if they realized that the path was being accessed. These unexpected path accesses can be caused by a number of reasons:

URL-encoded paths

As we noticed in the screenshot above, the VMware Tools process **VGAuthService.exe** attempts to access the path **C:\Program%20Files\VMware\VMware%20Tools\VMware%20VGAuth\schemas\xmldsig-core-schema.xsd.** How might this happen? If a path containing spaces is URL-encoded, those spaces will be replaced with %20.

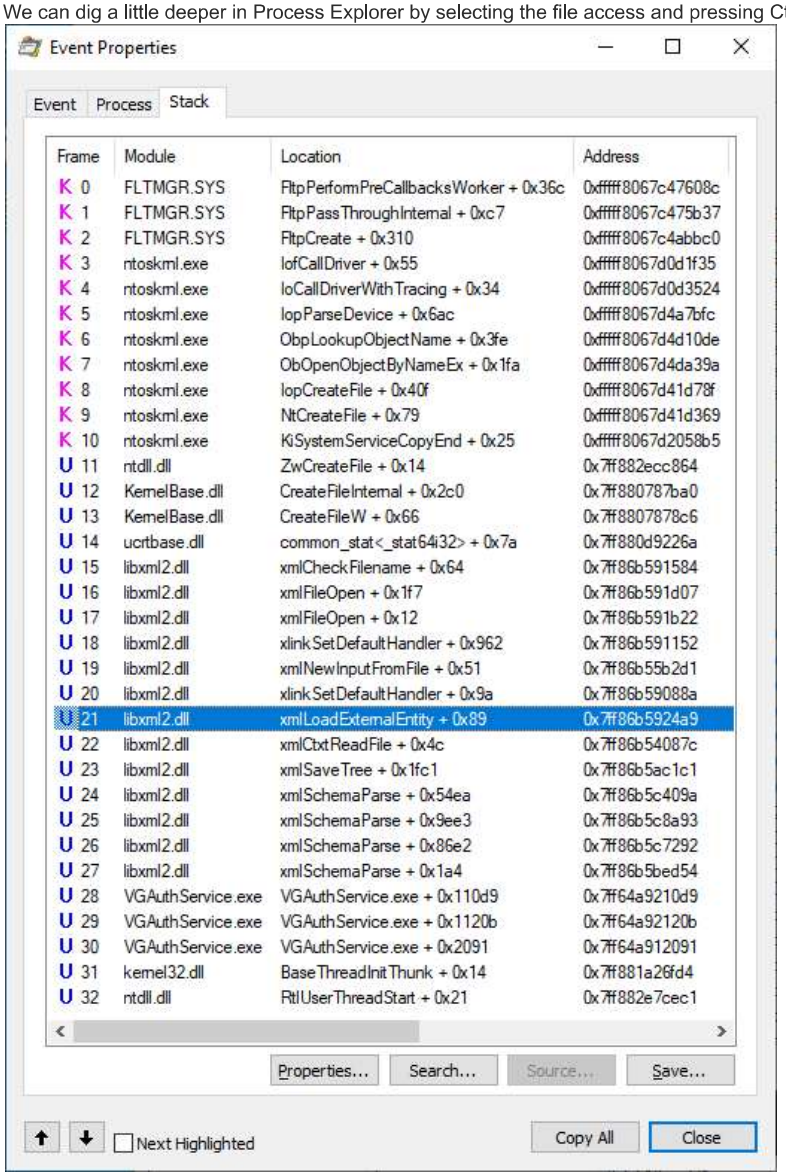
What are the consequences of this transformation? The most important aspect of this new path is that rather than being a subdirectory of **C:\Program Files**, which has proper ACLs by default, this requested path now starts looking at the root directory. Unprivileged users on Windows systems can create subdirectories off of the system root directory. This will be a recurring theme, so remember this.

From an unprivileged command prompt, let's see what we can do:



Success!

We can dig a little deeper in Process Explorer by selecting the file access and pressing Ctrl-K to get the call stack:



Frame	Module	Location	Address
K 0	FLTMGR.SYS	FltpPerformPreCallbacksWorker + 0x36c	0xfffff8067c47608c
K 1	FLTMGR.SYS	FltpPassThroughInternal + 0xc7	0xfffff8067c475b37
K 2	FLTMGR.SYS	FltpCreate + 0x310	0xfffff8067c4abbc0
K 3	ntoskrnl.exe	IofCallDriver + 0x55	0xfffff8067d0d1f35
K 4	ntoskrnl.exe	IoCallDriverWithTracing + 0x34	0xfffff8067d0d3524
K 5	ntoskrnl.exe	IopParseDevice + 0x6ac	0xfffff8067d4a7bfc
K 6	ntoskrnl.exe	ObpLookupObjectName + 0x3fe	0xfffff8067d4d10de
K 7	ntoskrnl.exe	ObOpenObjectByNameEx + 0x1fa	0xfffff8067d4da39a
K 8	ntoskrnl.exe	IopCreateFile + 0x40f	0xfffff8067d41d78f
K 9	ntoskrnl.exe	NtCreateFile + 0x79	0xfffff8067d41d369
K 10	ntoskrnl.exe	KiSystemServiceCopyEnd + 0x25	0xfffff8067d2058b5
U 11	ntdll.dll	ZwCreateFile + 0x14	0x7ff882ecc864
U 12	KernelBase.dll	CreateFileInternal + 0x2c0	0x7ff880787ba0
U 13	KernelBase.dll	CreateFileW + 0x66	0x7ff8807878c6
U 14	ucrtbase.dll	common_stat<_stat64i32> + 0x7a	0x7ff880d9226a
U 15	libxml2.dll	xmlCheckFilename + 0x64	0x7ff86b591584
U 16	libxml2.dll	xmlFileOpen + 0x1f7	0x7ff86b591d07
U 17	libxml2.dll	xmlFileOpen + 0x12	0x7ff86b591b22
U 18	libxml2.dll	xlinkSetDefaultHandler + 0x962	0x7ff86b591152
U 19	libxml2.dll	xmlNewInputFromFile + 0x51	0x7ff86b55b2d1
U 20	libxml2.dll	xlinkSetDefaultHandler + 0x9a	0x7ff86b59088a
U 21	libxml2.dll	xmlLoadExternalEntity + 0x89	0x7ff86b5924a9
U 22	libxml2.dll	xmlCtxtReadFile + 0x4c	0x7ff86b54087c
U 23	libxml2.dll	xmlSaveTree + 0x1fc1	0x7ff86b5ac1c1
U 24	libxml2.dll	xmlSchemaParse + 0x54ea	0x7ff86b5c409a
U 25	libxml2.dll	xmlSchemaParse + 0x9ee3	0x7ff86b5c8a93
U 26	libxml2.dll	xmlSchemaParse + 0x86e2	0x7ff86b5c7292
U 27	libxml2.dll	xmlSchemaParse + 0x1a4	0x7ff86b5bed54
U 28	VGAuthService.exe	VGAuthService.exe + 0x110d9	0x7ff64a9210d9
U 29	VGAuthService.exe	VGAuthService.exe + 0x1120b	0x7ff64a92120b
U 30	VGAuthService.exe	VGAuthService.exe + 0x2091	0x7ff64a912091
U 31	kernel32.dll	BaseThreadInitThunk + 0x14	0x7ff881a26fd4
U 32	ntdll.dll	RtlUserThreadStart + 0x21	0x7ff882e7cec1

Here we can see that the file access is triggered by **VGAuthService.exe + 0x110d9**, and along the way there is a call to **xmlLoadExternalEntity().**

Putting all of the pieces together here, we have a privileged process that attempts to load a file that does not exist because the path is URL encoded. Since an unprivileged user can create this path, this now turns into a case where an unprivileged user can influence a privileged process. In this particular case, the consequences are only an XML External Entity (XXE) vulnerability. But we're also just getting warmed up.

POSIX paths

If an application uses a POSIX-style path on a Windows machine, this path is normalized to a Windows style path. For example, if a Windows application attempts to access the **/usr/local/** directory, the path will be interpreted as **C:\usr\local**. And as described above, this is a path that an unprivileged user can create on Windows.

Here is a Process Monitor log of a system with a fully-patched security product installed:

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time ...	PID	Operation	Path	Result	Detail	Integrity	Arch...
10:38:...	3456	CreateFile	C:\usr\local\ssl\openssl.cnf	PATH NOT FOUND	Desired Access: G...	System	64-bit
10:38:...	3456	CreateFile	C:\usr\local\ssl\openssl.cnf	PATH NOT FOUND	Desired Access: G...	System	64-bit

Showing 198 of 5,476,640 events (0.0036%) Backed by C:\Users\test_user\Documents\Bootlog-6.pml

Using a publicly-known technique for achieving code execution via openssl.cnf, we can now demonstrate code execution via running **calc.exe** with SYSTEM privileges from a limited user account:

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time ...	PID	Operation	Path	Result	Detail	Integrity	Architectu
6:40:4...	3516	ReadFile	C:\usr\local\ssl\openssl.cnf	END OF FILE	Offset: 205, Length: 4,096	System	64-bit
6:40:4...	3516	CloseFile	C:\usr\local\ssl\openssl.cnf	SUCCESS		System	64-bit
6:40:4...	3516	CreateFile	C:\tmp\main.dll	SUCCESS	Desired Access: Read At...	System	64-bit
6:40:4...	3516	QueryBasicInfor...	C:\tmp\main.dll	SUCCESS	CreationTime: 8/4/2020 ...	System	64-bit
6:40:4...	3516	CloseFile	C:\tmp\main.dll	SUCCESS		System	64-bit
6:40:4...	3516	CreateFile	C:\tmp\main.dll	SUCCESS	Desired Access: Read D...	System	64-bit
6:40:4...	3516	CreateFileMapp...	C:\tmp\main.dll	FILE LOCKED W...	SyncType: SyncTypeCre...	System	64-bit
6:40:4...	3516	QueryStandardI...	C:\tmp\main.dll	SUCCESS	AllocationSize: 307,200, ...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 0, Length: 4,096, ...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 15,872, Length: 5...	System	64-bit
6:40:4...	3516	CreateFileMapp...	C:\tmp\main.dll	SUCCESS	SyncType: SyncTypeOther	System	64-bit
6:40:4...	3516	Load Image	C:\tmp\main.dll	SUCCESS	Image Base: 0x5308000...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 12,800, Length: 2...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 15,360, Length: 5...	System	64-bit
6:40:4...	3516	CloseFile	C:\tmp\main.dll	SUCCESS		System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 14,848, Length: 5...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 10,752, Length: 1...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 11,776, Length: 5...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 1,536, Length: 4,...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 1,536, Length: 7,...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 9,216, Length: 1,...	System	64-bit
6:40:4...	3516	ReadFile	C:\tmp\main.dll	SUCCESS	Offset: 8,704, Length: 51...	System	64-bit
6:40:4...	3516	CreateFile	C:\Windows\System32\calc.exe	SUCCESS	Desired Access: Read At...	System	64-bit
6:40:4...	3516	QueryBasicInfor...	C:\Windows\System32\calc.exe	SUCCESS	CreationTime: 3/19/201...	System	64-bit
6:40:4...	3516	CloseFile	C:\Windows\System32\calc.exe	SUCCESS		System	64-bit
6:40:4...	3516	CreateFile	C:\Windows\System32\calc.exe	SUCCESS	Desired Access: Read At...	System	64-bit
6:40:4...	3516	QueryBasicInfor...	C:\Windows\System32\calc.exe	SUCCESS	CreationTime: 3/19/201...	System	64-bit
6:40:4...	3516	CloseFile	C:\Windows\System32\calc.exe	SUCCESS		System	64-bit
6:40:4...	3516	CreateFile	C:\Windows\System32\calc.exe	SUCCESS	Desired Access: Read D...	System	64-bit
6:40:4...	3516	CreateFileMapp...	C:\Windows\System32\calc.exe	FILE LOCKED W...	SyncType: SyncTypeCre...	System	64-bit
6:40:4...	3516	QueryStandardI...	C:\Windows\System32\calc.exe	SUCCESS	AllocationSize: 28,672, E...	System	64-bit
6:40:4...	3516	ReadFile	C:\Windows\System32\calc.exe	SUCCESS	Offset: 0, Length: 27,648...	System	64-bit
6:40:4...	3516	ReadFile	C:\Windows\System32\calc.exe	SUCCESS	Offset: 27,136, Length: 5...	System	64-bit
6:40:4...	3516	CreateFileMapp...	C:\Windows\System32\calc.exe	SUCCESS	SyncType: SyncTypeOther	System	64-bit
6:40:4...	3516	QueryNameInfo...	C:\Windows\System32\calc.exe	SUCCESS	Name: \Windows\Syste...	System	64-bit
6:40:4...	3516	Process Create	C:\WINDOWS\SYSTEM32\calc.exe	SUCCESS	PID: 2076, Command lin...	System	64-bit
6:40:4...	3516	QuerySecurityFile	C:\Windows\System32\calc.exe	SUCCESS	Information: Owner, Grou...	System	64-bit
6:40:4...	3516	QueryBasicInfor...	C:\Windows\System32\calc.exe	SUCCESS	CreationTime: 3/19/201...	System	64-bit
6:40:4...	3516	CloseFile	C:\Windows\System32\calc.exe	SUCCESS		System	64-bit
6:40:4...	2076	Load Image	C:\Windows\System32\calc.exe	SUCCESS	Image Base: 0x7ff7556c...	System	64-bit
6:40:4...	2076	QueryNameInfo...	C:\Windows\System32\calc.exe	SUCCESS	Name: \Windows\Syste...	System	64-bit
6:40:4...	2076	QueryNameInfo...	C:\Windows\System32\calc.exe	SUCCESS	Name: \Windows\Syste...	System	64-bit
6:40:4...	3516	CreateFile	C:\Windows\System32\calc.exe	SUCCESS	Desired Access: Read At...	System	64-bit

Showing 77 of 3,051,050 events (0.0025%) Backed by C:\tmp\Downloads\Bootlog-calc-3.pml

Use of a library that loads from an unexpected path

In some cases, a developer may have done nothing wrong other than using a library that happens to have load from a location that can be influenced by an unprivileged Windows user. For example, here's a Process Monitor log of an application that attempts to access the path **C:\CMU\bin\sasl2**:

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time ...	PID	Operation	Path	Result	Detail	Integrity	Arch...
6:47:2...	3952	CreateFile	C:\CMU\bin\sasl2	PATH NOT FOUND	Desired Access: R...	System	64-bit

Showing 80 of 1,186,341 events (0.0067%) Backed by virtual memory

If we look at the call stack, we can see that this access is likely triggered by the **libsasl.dll** library:

Event Properties

Event | Process | Stack

Frame	Module	Location	Address
K 0	FLTMGR.SYS	FltpPerformPreCallbacks + 0x2fd	0xfffff8077f534a5d
K 1	FLTMGR.SYS	FltpPassThroughInternal + 0x90	0xfffff8077f5345a0
K 2	FLTMGR.SYS	FltpCreate + 0x2f3	0xfffff8077f56cd13
K 3	ntoskrnl.exe	IofCallDriver + 0x59	0xfffff8077e53ded9
K 4	ntoskrnl.exe	IoCallDriverWithTracing + 0x34	0xfffff8077e53cf94
K 5	ntoskrnl.exe	IopParseDevice + 0x62b	0xfffff8077eaa8cab
K 6	ntoskrnl.exe	ObpLookupObjectName + 0x78f	0xfffff8077eaafc7f
K 7	ntoskrnl.exe	ObOpenObjectByNameEx + 0x201	0xfffff8077eaae0e1
K 8	ntoskrnl.exe	IopCreateFile + 0x820	0xfffff8077eaf5910
K 9	ntoskrnl.exe	NtCreateFile + 0x79	0xfffff8077eaf50d9
K 10	ntoskrnl.exe	KiSystemServiceCopyEnd + 0x25	0xfffff8077e677c15
U 11	ntdll.dll	ZwCreateFile + 0x14	0x7ffecddfcb14
U 12	KernelBase.dll	CreateFileInternal + 0x2f4	0x7ffecbb344c4
U 13	KernelBase.dll	CreateFileW + 0x66	0x7ffecbb341b6
U 14	ucrtbase.dll	common_stat<_stat64i32> + 0xa0	0x7ffecb92ef54
U 15	ucrtbase.dll	stat64i32 + 0x76	0x7ffecb930386
U 16	libsasl.dll	sasl_seterror + 0x980	0x7ffeb5c42ce0
U 17	libsasl.dll	sasl_client_init + 0x1a7	0x7ffeb5c36de7

And sure enough, if we look at the code for libsasl, we can see a hard-coded reference to the path **C:\CMU\bin\sasl2**.

As an unprivileged user, we can create the directory and place whatever code we want there. Once again, we have calc.exe executing with SYSTEM privileges. All from an unprivileged user account.

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time ...	PID	Operation	Path	Result	Detail	Integrity	Arch...
2:13:0...	96	CloseFile	C:\CMU\bin\sasl2	SUCCESS		System	64-bit
2:13:0...	96	CreateFile	C:\CMU\bin\sasl2	SUCCESS	Desired Access: R...	System	64-bit
2:13:0...	96	QueryDirectory	C:\CMU\bin\sasl2*.dll	SUCCESS	Filter: *.dll, 1: main6...	System	64-bit
2:13:0...	96	CreateFile	C:\CMU\bin\sasl2\main64.dll	SUCCESS	Desired Access: R...	System	64-bit
2:13:0...	96	QueryBasicInfor...	C:\CMU\bin\sasl2\main64.dll	SUCCESS	CreationTime: 10/7...	System	64-bit
2:13:0...	96	CloseFile	C:\CMU\bin\sasl2\main64.dll	SUCCESS		System	64-bit
2:13:0...	96	CreateFile	C:\CMU\bin\sasl2\main64.dll	SUCCESS	Desired Access: R...	System	64-bit
2:13:0...	96	QueryEAFile	C:\CMU\bin\sasl2\main64.dll	SUCCESS		System	64-bit
2:13:0...	2424	CreateFileMapp...	C:\CMU\bin\sasl2\main64.dll	FILE LOCKED...	SyncType: SyncTy...	System	64-bit
2:13:0...	2424	QueryStandardI...	C:\CMU\bin\sasl2\main64.dll	SUCCESS	AllocationSize: 307...	System	64-bit
2:13:0...	96	CreateFileMapp...	C:\CMU\bin\sasl2\main64.dll	FILE LOCKED...	SyncType: SyncTy...	System	64-bit
2:13:0...	96	QueryStandardI...	C:\CMU\bin\sasl2\main64.dll	SUCCESS	AllocationSize: 307...	System	64-bit
2:13:0...	96	CreateFileMapp...	C:\CMU\bin\sasl2\main64.dll	SUCCESS	SyncType: SyncTy...	System	64-bit
2:13:0...	96	Load Image	C:\CMU\bin\sasl2\main64.dll	SUCCESS	Image Base: 0x5b5...	System	64-bit
2:13:0...	96	CloseFile	C:\CMU\bin\sasl2\main64.dll	SUCCESS		System	64-bit
2:13:0...	96	CreateFile	C:\Windows\System32\calc.exe	SUCCESS	Desired Access: R...	System	64-bit
2:13:0...	96	QueryBasicInfor...	C:\Windows\System32\calc.exe	SUCCESS	CreationTime: 3/19...	System	64-bit

Showing 414 of 487,669 events (0.084%) | Backed by virtual memory

Use of paths that only exist on a developer's system

Sometimes a program may contain references to paths that only exist on the developer's system. As long as the software functions properly on systems that do not have such a directory, then this attribute may not be recognized unless somebody is looking. For example, this software looks for a plugins subdirectory in the **C:\Qt** directory:

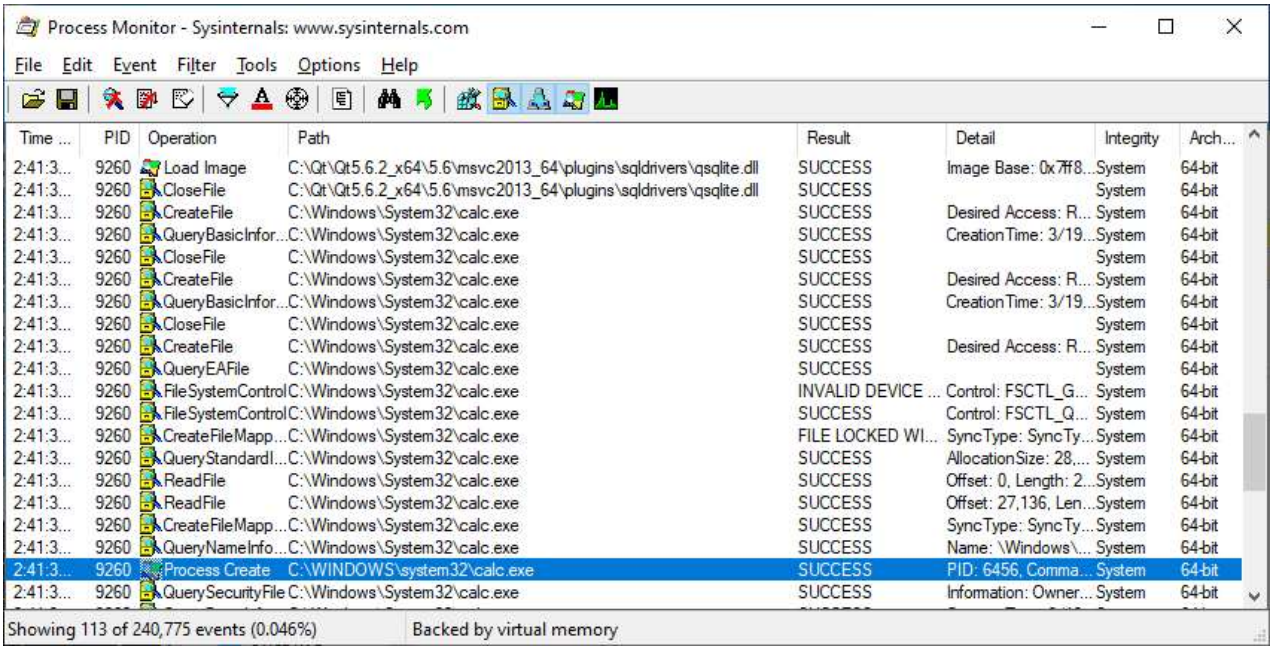
Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time ...	PID	Operation	Path	Result	Detail	Integrity	Arch...
2:28:0...	5872	CreateFile	C:\Qt\Qt5.6.2_x64\5.6\msvc2013_64\plugins	PATH NOT FOUND	Desired Access: R...	System	64-bit

Showing 4 of 859,359 events (0.00046%) | Backed by virtual memory

I'll skip some steps for the sake of brevity, but after a bit of investigation we see that we can achieve code execution by placing a special library in the appropriate directory:



Looking further into the Qt development platform, this type of vulnerability is a known issue. The vulnerability was patched more than 5 years ago, but it never received a CVE. Software may be vulnerable to privilege escalation if it was built with a Qt version from before this patch was introduced or the developer did not use windeployqt to patch out the **qt_prfxpath** value stored in **Qt5core.dll**.

Unexpected ACLs applied to paths being used

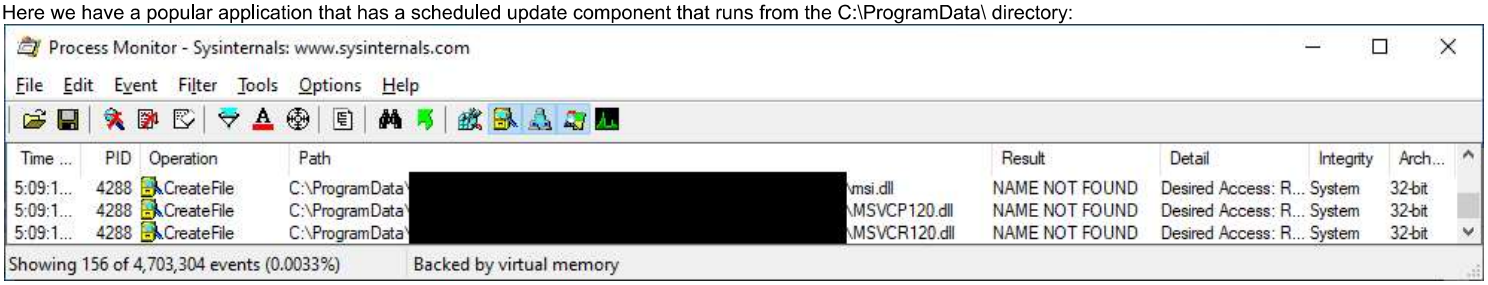
Most cases of an unexpected path being accessed by an application can be exploited because of a simple fact: unprivileged users can create subdirectories off of the Windows system root directory. Finding and exploiting software that fails to properly set ACLs requires just a bit more investigation.

Most of the ACL issues related to Windows software is related to one concept:
Software that executes from a subdirectory of **C:\Program Files** or **C:\Program Files (x86)** has secure ACLs **by default** by virtue of inheritance. For example, consider the case where I install my software to **C:\Program Files\WD**. Unprivileged users will not be able to modify the contents of the WD subdirectory because its parent directory of **C:\Program Files** cannot be written to by unprivileged processes, and the **WD** subdirectory by default will inherit its parents permissions.

Using the C:\ProgramData\ directory without explicitly settings ACLs

The ProgramData directory by design can be written to without elevated permissions. As such, any subdirectory that has been created in the ProgramData directory will by default be writable by unprivileged users. Depending on **how** an application uses its ProgramData subdirectory, a privilege escalation may be possible if the ACLs for the subdirectory are not explicitly set.

Here we have a popular application that has a scheduled update component that runs from the C:\ProgramData\ directory:



This is a straightforward potential case of DLL hijacking, which is made possible due to lax ACLs on the directory from which the software runs. Let's plant a crafted msi.dll there and see what we can accomplish:

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time ...	PID	Operation	Path	Result	Detail	Integrity	Arch...
5:08:5...	6332	Load Image	C:\ProgramData\[redacted]msi.dll	SUCCESS	Image Base: 0x708...	System	32-bit
5:08:5...	6332	CloseFile	C:\ProgramData\[redacted]msi.dll	SUCCESS		System	32-bit
5:08:5...	6332	CreateFile	C:\ProgramData\[redacted]msi.dll	SUCCESS	Desired Access: R...	System	32-bit
5:08:5...	6332	QuerySecurityFile	C:\ProgramData\[redacted]msi.dll	BUFFER OVERFL...	Information: Owner	System	32-bit
5:08:5...	6332	QuerySecurityFile	C:\ProgramData\[redacted]msi.dll	SUCCESS	Information: Owner	System	32-bit
5:08:5...	6332	CloseFile	C:\ProgramData\[redacted]msi.dll	SUCCESS		System	32-bit
5:08:5...	6332	CreateFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Desired Access: R...	System	32-bit
5:08:5...	6332	QueryBasicInfor...	C:\Windows\SysWOW64\calc.exe	SUCCESS	CreationTime: 3/19...	System	32-bit
5:08:5...	6332	CloseFile	C:\Windows\SysWOW64\calc.exe	SUCCESS		System	32-bit
5:08:5...	6332	CreateFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Desired Access: R...	System	32-bit
5:08:5...	6332	QueryBasicInfor...	C:\Windows\SysWOW64\calc.exe	SUCCESS	CreationTime: 3/19...	System	32-bit
5:08:5...	6332	CloseFile	C:\Windows\SysWOW64\calc.exe	SUCCESS		System	32-bit
5:08:5...	6332	CreateFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Desired Access: R...	System	32-bit
5:08:5...	6332	QueryEAFile	C:\Windows\SysWOW64\calc.exe	SUCCESS		System	32-bit
5:08:5...	6332	FileSystemControl	C:\Windows\SysWOW64\calc.exe	INVALID DEVICE ...	Control: FSCTL_G...	System	32-bit
5:08:5...	6332	FileSystemControl	C:\Windows\SysWOW64\calc.exe	SUCCESS	Control: FSCTL_Q...	System	32-bit
5:08:5...	6332	CreateFileMapp...	C:\Windows\SysWOW64\calc.exe	FILE LOCKED WI...	SyncType: SyncTy...	System	32-bit
5:08:5...	6332	QueryStandardI...	C:\Windows\SysWOW64\calc.exe	SUCCESS	AllocationSize: 28,...	System	32-bit
5:08:5...	6332	ReadFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Offset: 0, Length: 2...	System	32-bit
5:08:5...	6332	ReadFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Offset: 25,600, Len...	System	32-bit
5:08:5...	6332	CreateFileMapp...	C:\Windows\SysWOW64\calc.exe	SUCCESS	SyncType: SyncTy...	System	32-bit
5:08:5...	6332	QueryNameInfo...	C:\Windows\SysWOW64\calc.exe	SUCCESS	Name: \Windows\...	System	32-bit
5:08:5...	6332	Process Create	C:\WINDOWS\SysWOW64\calc.exe	SUCCESS	PID: 5696, Comma...	System	32-bit
5:08:5...	6332	QuerySecurityFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Information: Owner...	System	32-bit

Showing 4,805 of 6,262,590 events (0.076%) Backed by virtual memory

There's our calc.exe, executing with SYSTEM privileges. These problems seem a bit too prevalent. And easy to exploit.

It's worth noting that DLL hijacking isn't our only option for privilege escalation. **Any** user-writable file that is used by a privileged process introduces the possibility of introducing a privilege escalation vulnerability. For example, here's a popular program that checks for a user-creatable text file to direct its privileged auto-update mechanism. As we can see here, the presence of a crafted text file can lead to arbitrary command execution. In our case, we have it launch calc.exe:

Process Monitor - Sysinternals: www.sysinternals.com

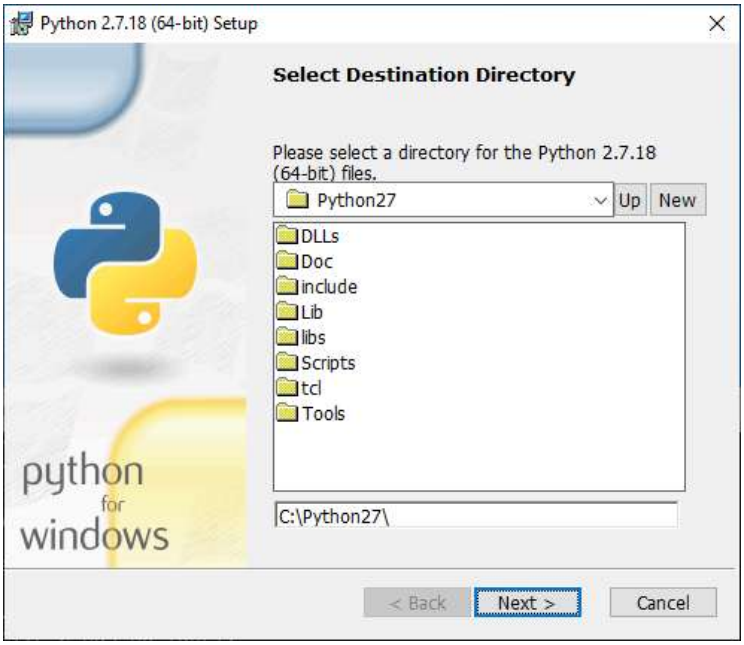
File Edit Event Filter Tools Options Help

Time ...	PID	Operation	Path	Result	Detail	Integrity	Arch...
5:03:0...	7508	CreateFile	C:\ProgramData\[redacted].txt	SUCCESS	Desired Access: R...	System	32-bit
5:03:0...	7508	QueryAttributeT...	C:\ProgramData\[redacted].txt	SUCCESS	Attributes: A, Repa...	System	32-bit
5:03:0...	7508	SetDispositionI...	C:\ProgramData\[redacted].txt	SUCCESS	Flags: FILE_DISP...	System	32-bit
5:03:0...	7508	CloseFile	C:\ProgramData\[redacted].txt	SUCCESS		System	32-bit
5:03:0...	7508	CreateFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Desired Access: R...	System	32-bit
5:03:0...	7508	QueryBasicInfor...	C:\Windows\SysWOW64\calc.exe	SUCCESS	CreationTime: 3/19...	System	32-bit
5:03:0...	7508	CloseFile	C:\Windows\SysWOW64\calc.exe	SUCCESS		System	32-bit
5:03:0...	7508	CreateFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Desired Access: R...	System	32-bit
5:03:0...	7508	QueryBasicInfor...	C:\Windows\SysWOW64\calc.exe	SUCCESS	CreationTime: 3/19...	System	32-bit
5:03:0...	7508	CloseFile	C:\Windows\SysWOW64\calc.exe	SUCCESS		System	32-bit
5:03:0...	7508	CreateFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Desired Access: R...	System	32-bit
5:03:0...	7508	QueryEAFile	C:\Windows\SysWOW64\calc.exe	SUCCESS		System	32-bit
5:03:0...	7508	FileSystemControl	C:\Windows\SysWOW64\calc.exe	INVALID DEVI...	Control: FSCTL_G...	System	32-bit
5:03:0...	7508	FileSystemControl	C:\Windows\SysWOW64\calc.exe	SUCCESS	Control: FSCTL_Q...	System	32-bit
5:03:0...	7508	CreateFileMapp...	C:\Windows\SysWOW64\calc.exe	FILE LOCKED ...	SyncType: SyncTy...	System	32-bit
5:03:0...	7508	QueryStandardI...	C:\Windows\SysWOW64\calc.exe	SUCCESS	AllocationSize: 28,...	System	32-bit
5:03:0...	7508	ReadFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Offset: 0, Length: 2...	System	32-bit
5:03:0...	7508	ReadFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Offset: 25,600, Len...	System	32-bit
5:03:0...	7508	CreateFileMapp...	C:\Windows\SysWOW64\calc.exe	SUCCESS	SyncType: SyncTy...	System	32-bit
5:03:0...	7508	QueryNameInfo...	C:\Windows\SysWOW64\calc.exe	SUCCESS	Name: \Windows\...	System	32-bit
5:03:0...	7508	Process Create	c:\windows\SysWOW64\calc.exe	SUCCESS	PID: 4620, Comma...	System	32-bit
5:03:0...	7508	QuerySecurityFile	C:\Windows\SysWOW64\calc.exe	SUCCESS	Information: Owner...	System	32-bit

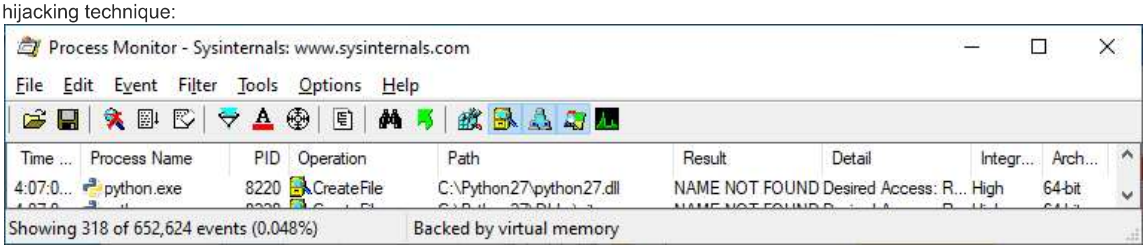
Showing 410 of 450,284 events (0.091%) Backed by virtual memory

Installing to a subdirectory off of the system root

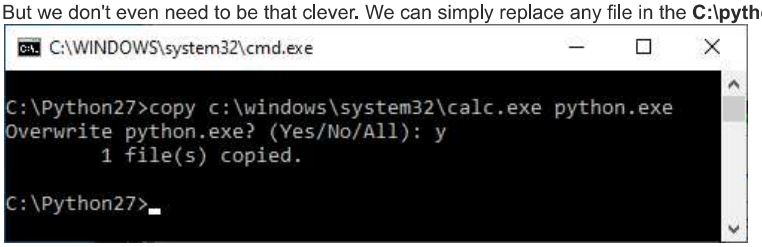
An installer that places an application by default to a directory off of the system root must set appropriate ACLs to remain secure. For example, Python 2.7 installs to **C:\python27** by default:



The default ACLs for this directory allow unprivileged users to modify the contents of this directory. What might we be able to do with this? We can try the standard DLL hijacking technique:



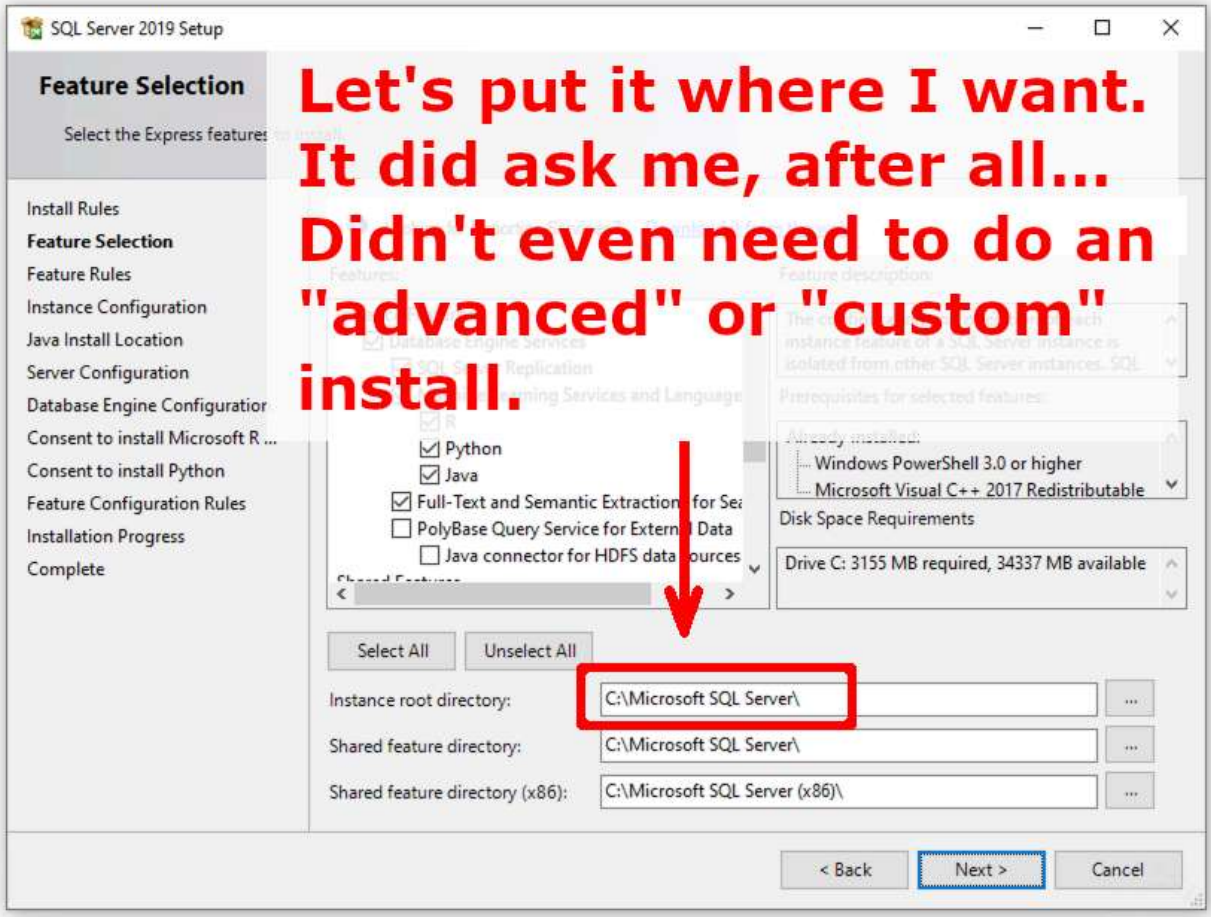
But we don't even need to be that clever. We can simply replace any file in the **C:\python27** directory as an unprivileged user:



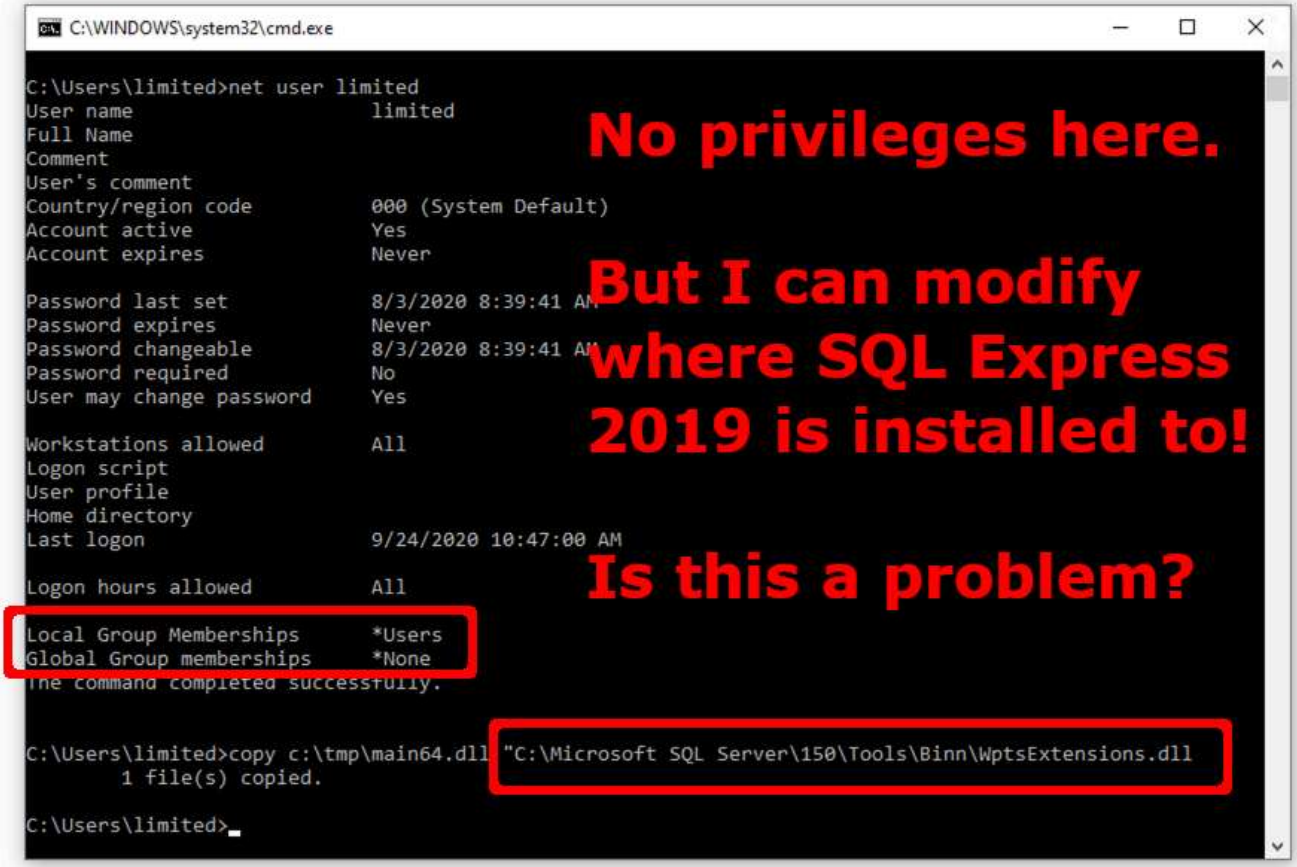
Allowing user-specified installation directories without settings ACLs

Many installers are secure because of inheritance of secure ACLs from C:\Program Files\. However any installer that allows a user to choose their own installation directory must explicitly set ACLs in the target location. Sadly, in my testing I've found that it is very rare for an installer to explicitly set ACLs. Let's take a look at the

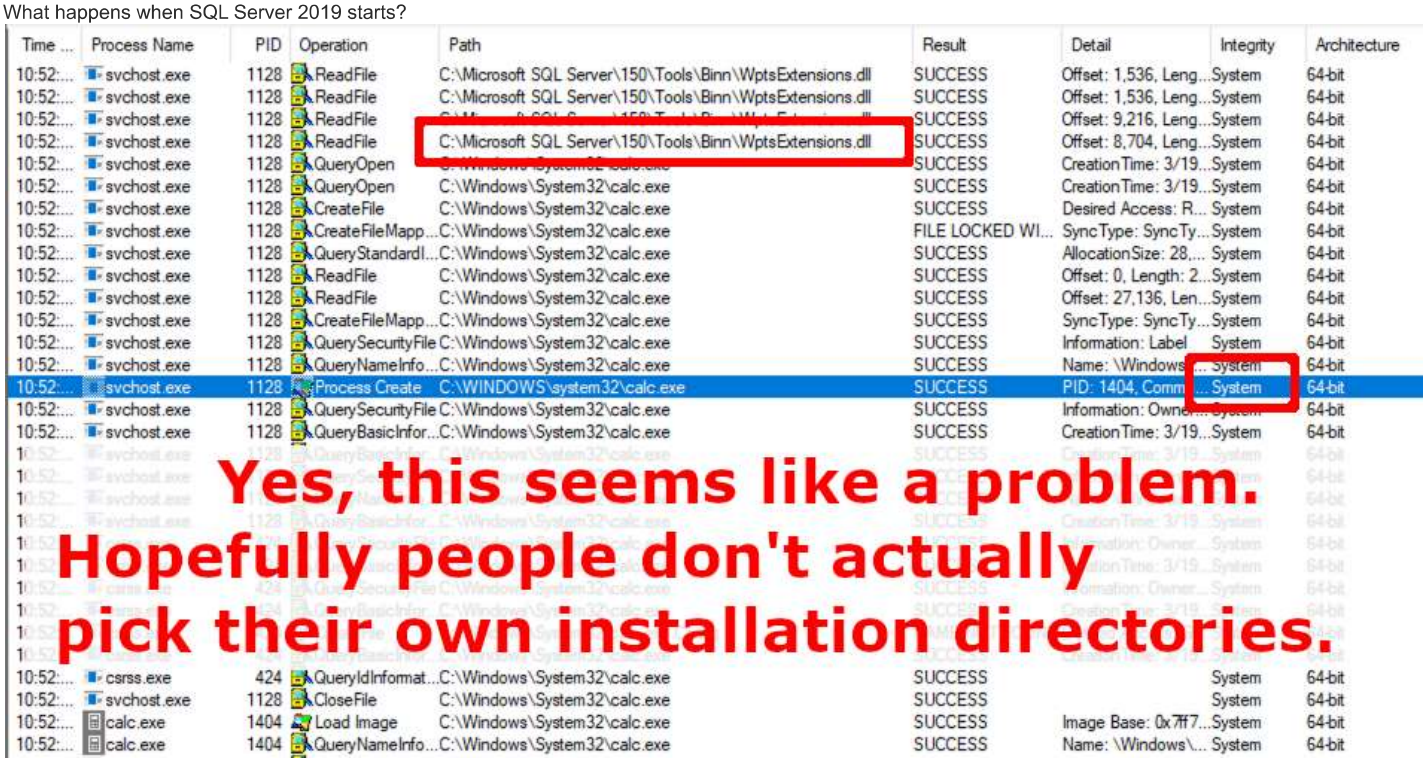
Microsoft SQL Server 2019 installer, for example:



Does the installer set ACLs to the directory where it installs the software?



What happens when SQL Server 2019 starts?



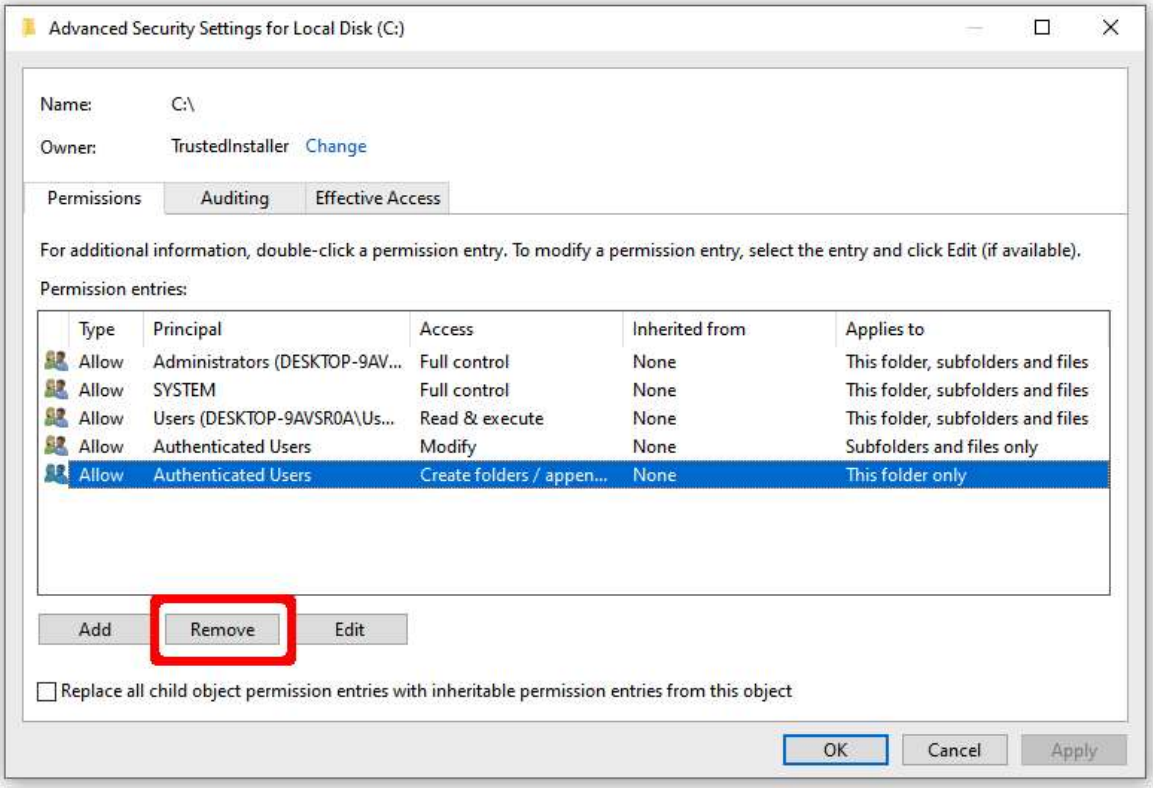
Time ...	Process Name	PID	Operation	Path	Result	Detail	Integrity	Architecture
10:52:...	svchost.exe	1128	ReadFile	C:\Microsoft SQL Server\150\Tools\Binn\WptsExtensions.dll	SUCCESS	Offset: 1,536, Leng...	System	64-bit
10:52:...	svchost.exe	1128	ReadFile	C:\Microsoft SQL Server\150\Tools\Binn\WptsExtensions.dll	SUCCESS	Offset: 1,536, Leng...	System	64-bit
10:52:...	svchost.exe	1128	ReadFile	[illegible]	SUCCESS	Offset: 9,216, Leng...	System	64-bit
10:52:...	svchost.exe	1128	ReadFile	C:\Microsoft SQL Server\150\Tools\Binn\WptsExtensions.dll	SUCCESS	Offset: 8,704, Leng...	System	64-bit
10:52:...	svchost.exe	1128	QueryOpen	[illegible]	SUCCESS	Creation Time: 3/19...	System	64-bit
10:52:...	svchost.exe	1128	QueryOpen	C:\Windows\System32\calc.exe	SUCCESS	Creation Time: 3/19...	System	64-bit
10:52:...	svchost.exe	1128	CreateFile	C:\Windows\System32\calc.exe	SUCCESS	Desired Access: R...	System	64-bit
10:52:...	svchost.exe	1128	CreateFileMapp...	C:\Windows\System32\calc.exe	FILE LOCKED WI...	SyncType: SyncTy...	System	64-bit
10:52:...	svchost.exe	1128	QueryStandardI...	C:\Windows\System32\calc.exe	SUCCESS	Allocation Size: 28,...	System	64-bit
10:52:...	svchost.exe	1128	ReadFile	C:\Windows\System32\calc.exe	SUCCESS	Offset: 0, Length: 2...	System	64-bit
10:52:...	svchost.exe	1128	ReadFile	C:\Windows\System32\calc.exe	SUCCESS	Offset: 27,136, Len...	System	64-bit
10:52:...	svchost.exe	1128	CreateFileMapp...	C:\Windows\System32\calc.exe	SUCCESS	SyncType: SyncTy...	System	64-bit
10:52:...	svchost.exe	1128	QuerySecurityFile	C:\Windows\System32\calc.exe	SUCCESS	Information: Label	System	64-bit
10:52:...	svchost.exe	1128	QueryNameInfo...	C:\Windows\System32\calc.exe	SUCCESS	Name: \Windows...	System	64-bit
10:52:...	svchost.exe	1128	Process Create	C:\WINDOWS\system32\calc.exe	SUCCESS	PID: 1404, Comm...	System	64-bit
10:52:...	svchost.exe	1128	QuerySecurityFile	C:\Windows\System32\calc.exe	SUCCESS	Information: Owner...	System	64-bit
10:52:...	svchost.exe	1128	QueryBasicInfor...	C:\Windows\System32\calc.exe	SUCCESS	Creation Time: 3/19...	System	64-bit
10:52:...	csrss.exe	424	QueryIdInformat...	C:\Windows\System32\calc.exe	SUCCESS		System	64-bit
10:52:...	svchost.exe	1128	CloseFile	C:\Windows\System32\calc.exe	SUCCESS		System	64-bit
10:52:...	calc.exe	1404	Load Image	C:\Windows\System32\calc.exe	SUCCESS	Image Base: 0x7ff7...	System	64-bit
10:52:...	calc.exe	1404	QueryNameInfo...	C:\Windows\System32\calc.exe	SUCCESS	Name: \Windows\...	System	64-bit

Microsoft SQL Server 2019, as well as just about any Windows application that allows you to choose where to install it, might be vulnerable to privilege escalation simply based on what directory it is installed to.

Defenses against privilege escalation

Remove "Create folders" permission on system root for unprivileged users

The simplest defense against many of the attacks outlined above is to remove the permission to create folders off of the system root directory:



Do not install software outside of C:\Program Files\

If software is installed to any location other than C:\Program Files\ or C:\Program Files (x86)\, you are relying on the installer to explicitly set ACLs for it to be secure. You can avoid needing to make this leap of faith by only installing software to recommended program locations.

Test and fortify your own systems

You can test your own platforms for privilege escalation vulnerabilities using the Process Monitor filter and techniques described above. For any file locations that are determined to be insecure, you can manually lock down those directories so that unprivileged users cannot modify those locations. For any vulnerabilities that you discover, we recommend contacting the affected vendors to notify them of the vulnerabilities so that they can be fixed for everyone. In cases where the vendor communications are unproductive, the CERT/CC may be able to provide assistance.

No labels

Carnegie Mellon
University
Software Engineering
Institute
4500 Fifth Avenue
Pittsburgh, PA
15213-2612
412-268-5800

Contact Us

Office Locations | Additional Sites Directory | Legal | Privacy Notice | CMU Ethics Hotline | www.sei.cmu.edu

® CERT and CERT Coordination Center are registered in the U.S. Patent and Trademark Office by Carnegie Mellon University.

© 2021 Carnegie Mellon University